# Financial Literacy for Students

Porter Clevidence
Antonio Duran-Ramos
Parth Gajjar
Arpit Vora

May 6, 2025

# List of Contents

# List of Figures

# List of Tables

# Project Plan

## Introduction

This project aims to build a search engine that allows students to retrieve relevant documents from a financial dataset. It is part of an educational initiative to promote financial literacy. This enables students to better understand and handle real-world financial situations encountered during their professional lives. To achieve this goal we have designed and implemented an *Information Retrieval (IR)* system that processes textual data, builds an *index* over a document collection, and ranks documents in response to user queries.

## Team Roles and Responsibilities

### Porter Clevidence

- Implemented BIM search
- Implemented BM25 search
- Implemented Language Model retrieval
- Implemented VSM search
- Search algorithm selection

### Antonio Duran-Ramos

- Development environment setup
- Graphical interface design
- LaTeX report setup
-

### Parth Gajjar

### Arpit Vora

## Methods and Techniques

In our system design we chose to use a combination of `Docker` and `uv`, along with various Python libraries. Docker allowed us to standardize our development environment so that our search engine behaved identical, regardless of host machine. uv allowed us to formalize our project as it handles initializing Python virtual environments and it manages dependencies for reproducible Python environments. As for the Python libraries we utilized, `pandas` was used to read from the provided JSON files to create structured data frames. `scikit-learn` was used for english stop words and its functions: `TfidfVectorizer` and `cosine_similarity`. `rank-bm25` was used for its implementation of the BM25 ranking algorithm. `streamlit` was used to implement the graphical interface for our search engine. The `nltk` library was used for its implementation of the stemmer developed by Martin Porter.

## Data

The dataset provided consisted of a collection of financial documents, along with static queries and their corresponding relevance judgments.

## Data Description and Analysis

### Dataset Statistics

The dataset consists of a collection of $5,000$ financial text documents, 25 static queries, and corresponding relevance judgments. This dataset was provided in separate JSON files.

## Text Processing

### Surface Level Normalization Definition

Before tokenizing and any further text processing, the text undergoes a three-stage cleaning process to ensure variations in formatting do not create duplicate tokens in our index.

1. The `.lower()` function converts every character in the documents to lowercase.
2. We strip out any punctuation, including periods, commas, quotes, brackets, and special symbols using a regular expression. Specifically, `re.sub(r'[^\w\s]', ")`, which tells Python to find any character that is *not* a word character (numbers included) or a whitespace character, and replace it with nothing.
3. We again use a regular expression to target any sequence of one or more spaces to collapse them to a single, clean space. Additionally, we apply the `.strip()` function to this regular expression to trim any lingering spaces at the beginning and end of a document.

### Tokenizer Definition

Now that the text has been normalized, we call `.split()` to break the strings apart at every single space. This function call results in a structured list of tokens.

### Stopping Definition

At the same time the `.split()` function is tokenizing the text, we immediately check the current token against the `ENGLISH_STOP_WORDS` list provided by the `scikit-learn` library. If the token is a highly common word, (i.e., "the", "is", "at", etc.), the token is instantly discarded. This results in a final tokens list full of meaningful words.

### Stemming/Lemmatizing Definition

Finally, the text processing pipeline ends by looping through the tokens and it applies `NLTK`'s `PorterStemmer()`, which removes common English suffixes and prefixes. This process shrinks our vocabulary drastically.

## Index Construction

### Term Weighting Definition

### Index Design

## Retrieval Model

### Ranking

# Implementation

## Source Code

## Video Demonstration

## Discussion

# References

[1]

[2]

[3]